🦓 Zebraw

Zebraw is a lightweight and fast package for displaying code blocks with line numbers in typst, supporting code line highlighting. The term ***Zebraw*** is a combination of ***zebra*** and ***raw***, for the highlighted lines will be displayed in the code block like a zebra lines.

Starting

Import `zebraw` package by `#import "@preview/zebraw:0.4.0": *` then follow with `#show: zebraw` to start using zebraw in the simplest way. To manually display some specific code blocks in zebraw, you can use `#zebraw()` function:

```typst
```typ
#grid(
  columns: (1fr, 1fr),
  [Hello], [world!],
)
```

#zebraw(
  ```typ
  #grid(
    columns: (1fr, 1fr),
    [Hello], [world!],
  )
  ```
)

#show: zebraw

```typ
#grid(
  columns: (1fr, 1fr),
  [Hello], [world!],
)
```
```

```typ
#grid(
  columns: (1fr, 1fr),
  [Hello], [world!],
)
```

```typ
#grid(
  columns: (1fr, 1fr),
  [Hello], [world!],
)
```

```typ
#grid(
  columns: (1fr, 1fr),
  [Hello], [world!],
)
```

Features

Line Highlighting

You can highlight specific lines in the code block by passing the `highlight-lines` parameter to the `zebraw` function. The `highlight-lines` parameter can be a single line number or an array of line numbers.

```
#zebraw(
  // Single line number:
  highlight-lines: 2,
  ``` typst
  #grid(
    columns: (1fr, 1fr),
    [Hello], [world!],
  )
  ```
)

#zebraw(
  // Array of line numbers:
  highlight-lines: (6, 7) + range(9, 15),
  ``` typst
  = Fibonacci sequence
  The Fibonacci sequence is defined through the
  recurrence relation $F_n = F_{n-1} + F_{n-2}$.
  It can also be expressed in closed form:
  $ F_n = round(1 / sqrt(5) phi.alt^n), quad
    phi.alt = (1 + sqrt(5)) / 2 $

#let count = 8
#let nums = range(1, count + 1)
#let fib(n) = {
  if n <= 2 { 1 }
  else { fib(n - 1) + fib(n - 2) }
}

The first #count numbers of the sequence are:

#align(center, table(
  columns: count,
  ..nums.map(n => $F_#n$),
  ..nums.map(n => str(fib(n))),
))
```
```
#zebraw(
  highlight-lines: (
    1, [The Fibonacci sequence is defined through the recurrence relation $F_n = F_{n-1} + F_{n-2}$\
    It can also be expressed in closed form:, $ F_n = round(1 / sqrt(5) phi.alt^n), quad
    phi.alt = (1 + sqrt(5)) / 2 $],
    // Passing a range of line numbers in the array should begin with `..`
    ..range(9, 14),
    [1], [The first \#count numbers of the sequence.],
  ),
  ``` typst
  = Fibonacci sequence
  #let count = 8
  #let nums = range(1, count + 1)
  #let fib(n) = {
    if n <= 2 { 1 }
    else { fib(n - 1) + fib(n - 2) }
  }

  #align(center, table(
    columns: count,
    ..nums.map(n => $F_#n$),
    ..nums.map(n => str(fib(n))),
  ))
  ```
)
```
```
#zebraw(
  highlight-lines: (
    // Comments can only be passed when highlight-lines is an array, so at the end of the single element array, a comma is needed.
    4, [6, [The Fibonacci sequence is defined through the recurrence relation $F_n = F_{n-1} + F_{n-2}$]],
  ),
  comment-flag: "~~>",
  ``` typst
  = Fibonacci sequence
  #let count = 8
  #let nums = range(1, count + 1)
  #let fib(n) = {
    if n <= 2 { 1 }
    else { fib(n - 1) + fib(n - 2) }
  }

  #align(center, table(
    columns: count,
    ..nums.map(n => $F_#n$),
    ..nums.map(n => str(fib(n))),
  ))
  ```
)
```

### Header and Footer

Usually, the comments passing by a dictionary of line numbers and comments are used to add a header or footer to the code block:

```
#zebraw(
  highlight-lines: (
    ..range(8, 13),
    [12, [The first \#count numbers of the sequence.]],
  ),
  header: [**Fibonacci sequence**],
  ``` typst
  #let count = 8
  #let nums = range(1, count + 1)
  #let fib(n) = {
    if n <= 2 { 1 }
    else { fib(n - 1) + fib(n - 2) }
  }

  #align(center, table(
    columns: count,
    ..nums.map(n => $F_#n$),
    ..nums.map(n => str(fib(n))),
  ))
  ```
  footer: [The Fibonacci sequence is defined through the recurrence relation $F_n = F_{n-1} + F_{n-2}$],
)
```

### Language Tab

If `lang` is set to `true`, then there will be a language tab on the top right corner of the code block:

```
#zebraw(
  lang: true,
  ``` typst
  #grid(
    columns: (1fr, 1fr),
    [Hello], [world!],
  )
  ```
)
```

### Copyable

If `copyable` is set to `true`, line numbers will not be copied when copying exported code.

```
#zebraw(
  copyable: true,
  ``` typst
  #grid(
    columns: (1fr, 1fr),
    [Hello], [world!],
  )
  ```
)
```

For comparison:

copyable: false

```
#grid(
  columns: (1fr, 1fr),
  [Hello], [world!],
)
```

copyable: true

```
#grid(
  columns: (1fr, 1fr),
  [Hello], [world!],
)
```

However when a code block is `copyable`, it won't be able to cross page. Only line numbers will be excluded for being selected.

Theme

PRs are welcome!

```
#show: zebraw-init.with(..zebraw-themes.zebra, lang: false)
#show: zebraw
```
```
// rust
fn fibonacci_recursive(n: i32) -> u64 {
    if n < 0 {
        panic!("{} is negative!", n);
    }
    match n {
        0 => panic!("zero is not a right argument to fibonacci_recursive()!"),
        1 | 2 => 1,
        3 => 2,
        _ => fibonacci_recursive(n - 1) + fibonacci_recursive(n - 2),
    }
}
```
```
fn fibonacci_recursive(n: i32) -> u64 {
    if n < 0 {
        panic!("{} is negative!", n);
    }
    match n {
        0 => panic!("zero is not a right argument to fibonacci_recursive()!"),
        1 | 2 => 1,
        3 => 2,
        _ => fibonacci_recursive(n - 1) + fibonacci_recursive(n - 2),
    }
}
```

```
#show: zebraw-init.with(..zebraw-themes.zebra-reverse, lang: false)
#show: zebraw
```
```
// rust
fn fibonacci_recursive(n: i32) -> u64 {
    if n < 0 {
        panic!("{} is negative!", n);
    }
    match n {
        0 => panic!("zero is not a right argument to fibonacci_recursive()!"),
        1 | 2 => 1,
        3 => 2,
        _ => fibonacci_recursive(n - 1) + fibonacci_recursive(n - 2),
    }
}
```
```
fn fibonacci_recursive(n: i32) -> u64 {
    if n < 0 {
        panic!("{} is negative!", n);
    }
    match n {
        0 => panic!("zero is not a right argument to fibonacci_recursive()!"),
        1 | 2 => 1,
        3 => 2,
        _ => fibonacci_recursive(n - 1) + fibonacci_recursive(n - 2),
    }
}
```
```

Customization

There are 3 ways to customize code blocks in your document:

- Manually render some specific blocks by `#zebraw()` function and passing parameters to it.
- By passing parameters to `#show: zebraw.with()` will affect every raw block after the `#show` rule, **except** blocks created manually by `#zebraw()` function.
- By passing parameters to `#show: zebraw-init.with()` will affect every raw block after the `#show` rule, **including** blocks created manually by `#zebraw()` function. By using `zebraw-init` without any parameters, the values will be reset to default.

Inset

Customize the inset of each line by passing a `dictionary` to the `inset` parameter:

```
#zebraw(
  inset: (top: 6pt, bottom: 6pt),
  ```typ
  #grid(
    columns: (1fr, 1fr),
    [Hello], [world!],
  )
  ```
)
```

```
#grid(
  columns: (1fr, 1fr),
  [Hello], [world!],
)
```

Colors

Customize the background color by passing a `color` or an `array` of `colors` to the `background-color` parameter.

```
#zebraw(
  background-color: luma(235),
  ```typ
  #grid(
    columns: (1fr, 1fr),
    [Hello], [world!],
  )
  ```,
)

#zebraw(
  background-color: (luma(235), luma(245), luma(255)),
  ```typ
  #grid(
    columns: (1fr, 1fr),
    [Hello], [world!],
  )
  ```,
)
```

```
#grid(
  columns: (1fr, 1fr),
  [Hello], [world!],
)
```

```
#grid(
  columns: (1fr, 1fr),
  [Hello], [world!],
)
```

Customize the highlight color by passing a `color` to the `highlight-color` parameter:

```
#zebraw(
  highlight-lines: 1,
  highlight-color: blue.lighten(90%),
  ```text
  I'm so blue!
                -- George III
  ```,
)
```

```
I'm so blue!
                -- George III
```

Customize the comments' background color by passing a `color` to the `comment-color` parameter:

```
#zebraw(
  highlight-lines: (
    (2, "auto indent!"),
  ),
  comment-color: yellow.lighten(90%),
  ```text
  I'm so blue!
                -- George III
  I'm not.
                -- Hamilton
  ```,
)
```

```
I'm so blue!
                -- George III
                > auto indent!
I'm not.
                -- Hamilton
```

Customize the language tab's background color by passing a `color` to the `lang-color` parameter.

```
#zebraw(
  lang: true,
  lang-color: teal,
  ```typst
  #grid(
    columns: (1fr, 1fr),
    [Hello], [world!],
  )
  ```
)
```

typst

```
#grid(
  columns: (1fr, 1fr),
  [Hello], [world!],
)
```

Font

To customize the arguments of comments' font and the language tab's font, pass a dictionary to `comment-font-args` parameter and `lang-font-args` parameter.

Language tab will be rendered as comments if nothing is passed.

```
#zebraw(
  highlight-lines: (
    (2, "columns ... "),
  ),
  lang: true,
  comment-color: white,
  comment-font-args: (
    font: "IBM Plex Sans",
    style: "italic"
  ),
  ```typst
  #grid(
    columns: (1fr, 1fr),
    [Hello], [world!],
  )
  ```
)
```

typst

```
#grid(
  columns: (1fr, 1fr),
  > columns...
  [Hello], [world!],
)
```

```
#zebraw(
  highlight-lines: (
    (2, "columns ... "),
  ),
  lang: true,
  lang-color: eastern,
  lang-font-args: (
    font: "libertinus serif",
    weight: "bold",
    fill: white,
  ),
  comment-font-args: (
    font: "IBM Plex Sans",
    style: "italic"
  ),
  ```typst
  #grid(
    columns: (1fr, 1fr),
    [Hello], [world!],
  )
  ```
)
```

typst

```
#grid(
  columns: (1fr, 1fr),
  > columns...
  [Hello], [world!],
)
```

Extend

Extend at vertical is enabled at default. When there's header or footer it will be automatically disabled.

```
#zebraw(
  extend: false,
  ```typst
  #grid(
    columns: (1fr, 1fr),
    [Hello], [world!],
  )
  ```
)
```

```
#grid(
  columns: (1fr, 1fr),
  [Hello], [world!],
)
```

Documentation

The default value of most parameters are *none* for it will use the default value in `zebraw-init`.

- `zebraw-init`
- `zebraw()`

`zebraw-init`

Initialize the `zebraw` block in global.

Parameters

```
zebraw-init
  inset: dictionary,
  background-color: false| array,
  highlight-color: false,
  comment-color: false,
  lang-color: false,
  comment-flag: string| content,
  lang: boolean,
  comment-fonts-args: dictionary,
  lang-font-args: dictionary,
  extend: boolean,
  numbering: boolean,
  body: content
) -> content
```

`inset` `dictionary`

The inset of each line.

Default: `(top: 0.3em, right: 0.3em, bottom: 0.3em, left: 0.3em)`

`background-color` `false| array`

The background color of the block and normal lines.

Default: `lum(245)`

`highlight-color` `false|`

The background color of the highlighted lines.

Default: `rgb("#fea2d5").lighten(70%)`

`comment-color` `false|`

The background color of the comments. When it's set to *none*, it will be rendered in a lightened `highlight-color`.

Default: *none*

`lang-color` `false|`

The background color of the language tab. The color is set to *none* at default and it will be rendered in comments' color.

Default: *none*

`comment-flag` `string | content`

The flag at the beginning of comments.

Default: `";"`

`lang` `boolean`

Whether to show the language tab.

Default: `true`

`comment-font-args` `dictionary`

The arguments passed to comments' font.

Default: `()`

`lang-font-args` `dictionary`

The arguments passed to the language tab's font.

Default: `()`

`extend` `boolean`

Whether to extend the vertical spacing.

Default: `true`

`copyable` `boolean`

Whether to enable "copyable" feature.

Default: `false`

`body` `content`

The body

`zebraw`

Block of code with highlighted lines and comments.

```
zebraw
  highlight-lines: (
    1,
    (5, "lm -rf 1230129409/"),
    8,
  ),
  lang: true,
  numbering: "Calculate Fibonacci number using recursive function",
  inset:
    (top: 0.3em,right: 0.3em,bottom: 0.3em,left: 0.3em),
  extend: true,
  header: "Fibonacci sequence",
  footer: "",
  background-color: lum(245),
  highlight-color: rgb("#fea2d5").lighten(70%),
  comment-color: lum(230),
  lang-color: lum(230),
  comment-flag: ";",
  comment-font-args: (),
  lang-font-args: (),
  copyable: false,
  body: [
    ```
    Calculate Fibonacci number using recursive function
    pub fn fibonacci(i,recursive=false) -> int = {
      if n == 0 {
        panic("If is negative!", n);
    }
    match n {
      0 => panic("zero is not a right argument to fibonacci!",n),
      1 => 1,
      2 => 1,
      _ => fibonacci_recursive(n - 1) + fibonacci_recursive(n - 2),
      n => lm -rf 1230129409/
    }
    }

    ```
  ],
)
```

Parameters

```
zebraw
  highlight-lines: array| int,
  header: string| content,
  footer: string| content,
  inset: dictionary,
  background-color: false| array,
  highlight-color: false|,
  comment-color: false|,
  lang-color: false|,
  comment-flag: string| content,
  lang: boolean,
  comment-font-args: dictionary,
  lang-font-args: dictionary,
  extend: boolean,
  copyable: boolean,
  body: content
) -> content
```

`highlight-lines` `array | int`

Lines to highlight or comments to show.

```
zebraw
  highlight-lines: range(5, 7),
  header: "Fibonacci sequence",
  ```
  pub fn fibonacci(i,recursive=false) -> int = {
    if count == 0
    if count == range(1, count + 1)
    if count == range(1, count + 1)
    if fib(i) > 1
      if n == 0 { 0 }
      else { fib(n - 1) + fib(n - 2) }
    }

    #alignCenter, table(
      columns: count,
      ..num_map(n => F_n),
      ..num_map(n => str(fib(n)))
)
    ```
    Further: The Fibonacci sequence is defined through the recurrence relation $F_n = F_{n-1} + F_{n-2}$.
  ],
)
```

Default: `()`

`header` `string | content`

The header of the code block.

Default: *none*

`footer` `string | content`

The footer of the code block.

Default: *none*

`inset` `dictionary`

The inset of each line.

```
zebraw
  inset: (top: 0pt, bottom: 0pt),
  ```
  pub fn fibonacci(i,recursive=false) -> int = {
    if count == 0
    if count == range(1, count + 1)
    if count == range(1, count + 1)
    if fib(i) > 1
      if n == 0 { 0 }
      else { fib(n - 1) + fib(n - 2) }
    }

    #alignCenter, table(
      columns: count,
      ..num_map(n => F_n),
      ..num_map(n => str(fib(n)))
)
    ```
    Further: The Fibonacci sequence is defined through the recurrence relation $F_n = F_{n-1} + F_{n-2}$.
  ],
)
```

Default: *none*

`background-color` `false | array`

The background color of the block and normal lines.

```
zebraw
  background-color: (lum(240), lum(245), lum(240), lum(235)),
  ```
  pub fn fibonacci(i,recursive=false) -> int = {
    if count == 0
    if count == range(1, count + 1)
    if count == range(1, count + 1)
    if fib(i) > 1
      if n == 0 { 0 }
      else { fib(n - 1) + fib(n - 2) }
    }

    #alignCenter, table(
      columns: count,
      ..num_map(n => F_n),
      ..num_map(n => str(fib(n)))
)
    ```
    Further: The Fibonacci sequence is defined through the recurrence relation $F_n = F_{n-1} + F_{n-2}$.
  ],
)
```

Default: *none*

`highlight-color` `false|`

The background color of the highlighted lines.

Default: *none*

`comment-color` `false|`

The background color of the comments. The color is set to *none* at default and it will be rendered in a lightened `highlight-color`.

```
zebraw
  highlight-color: yellow.lighten(60%),
  comment-color: yellow.lighten(90%),
  ```
  pub fn fibonacci(i,recursive=false) -> int = {
    if count == 0
    if count == range(1, count + 1)
    if count == range(1, count + 1)
    if fib(i) > 1
      if n == 0 { 0 }
      else { fib(n - 1) + fib(n - 2) }
    }

    #alignCenter, table(
      columns: count,
      ..num_map(n => F_n),
      ..num_map(n => str(fib(n)))
)
    ```
    Further: The Fibonacci sequence is defined through the recurrence relation $F_n = F_{n-1} + F_{n-2}$.
  ],
)
```

Default: *none*

`lang-color` `false|`

The background color of the language tab. The color is set to *none* at default and it will be rendered in comments' color.

```
zebraw
  lang: true,
  lang-color: eastern,
  lang-font-args:
    (font: "libertinus serif",
    weight: "bold",
    fill: white
  ),
  header:
    ("Aprio0
    calculate: (fib, 10),
    ["Hello], ["world"],
    ),
  ),
  ```
  pub fn fibonacci(i,recursive=false) -> int = {
    if count == 0
    if count == range(1, count + 1)
    if count == range(1, count + 1)
    if fib(i) > 1
      if n == 0 { 0 }
      else { fib(n - 1) + fib(n - 2) }
    }

    #alignCenter, table(
      columns: count,
      ..num_map(n => F_n),
      ..num_map(n => str(fib(n)))
)
    ```
    Further: The Fibonacci sequence is defined through the recurrence relation $F_n = F_{n-1} + F_{n-2}$.
  ],
)
```

Default: *none*

`comment-flag` `string | content`

The flag. When the flag is set to `""`, the indentation before the flag will be disabled as well.

```
zebraw
  comment-flag: "",
  highlight-lines: (
    1,(1, [The Fibonacci sequence is defined through the recurrence relation $F_n = F_{n-1} + F_{n-2}$.
    ]),
    (1, [The first 8count numbers of the sequence.]),
    ),
    header: "Fibonacci sequence
    ```
    pub fn fibonacci(i,recursive=false) -> int = {
      if count == 0
      if count == range(1, count + 1)
      if count == range(1, count + 1)
      if fib(i) > 1
        if n == 0 { 0 }
        else { fib(n - 1) + fib(n - 2) }
      }

      #alignCenter, table(
        columns: count,
        ..num_map(n => F_n),
        ..num_map(n => str(fib(n)))
)
      ```
    ],
)
```

Default: *none*

`lang` `boolean`

Whether to show the language tab on the top-right corner of the code block.

```
zebraw
  lang: true,
  ```
  Aprio0
    calculate: (fib, 10),
    ["Hello], ["world"],
)
  ```
)
```

Default: *none*

`comment-font-args` `dictionary`

The arguments passed to comments' font.

Default: *none*

`lang-font-args` `dictionary`

The arguments passed to the language tab's font.

```
zebraw
  lang: true,
  comment-font-args: (font: "IBM Plex Sans", weight: "bold",
  ),
  lang-font-args: (font: "IBM Plex Sans",
  weight: "bold",
  fill: rgb("#ffffff")
  ),
  header: ("The Fibonacci sequence is defined through the recurrence relation $F_n = F_{n-1} + F_{n-2}$.
    ]),
    (1, [The first 8count numbers of the sequence.]),
    ),
    header:
      ("The Fibonacci sequence is defined through the recurrence relation $F_n = F_{n-1} + F_{n-2}$.
      ]),
      (1, [The first 8count numbers of the sequence.]),
      ),
      header: "Fibonacci sequence
      ```
      pub fn fibonacci(i,recursive=false) -> int = {
        if count == 0
        if count == range(1, count + 1)
        if count == range(1, count + 1)
        if fib(i) > 1
          if n == 0 { 0 }
          else { fib(n - 1) + fib(n - 2) }
        }

        #alignCenter, table(
          columns: count,
          ..num_map(n => F_n),
          ..num_map(n => str(fib(n)))
)
        ```
      ],
)
```

Default: *none*

`extend` `boolean`

Whether to extend the vertical spacing.

```
zebraw
  extend: false,
  ```
  Aprio0
    calculate: (fib, 10),
    ["Hello], ["world"],
)
  ```
)
```

Default: *none*

`copyable` `boolean`

Whether to enable "copyable" feature.

If *copyable* feature is enabled, lines of code are displayed together in a block to allow copying code in an export. However when this feature is enabled, the code block **cannot** be displayed across pages.

```
zebraw
  copyable: true,
  ```
  Aprio0
    calculate: (fib, 10),
    ["Hello], ["world"],
)
  ```
)
```

Default: *none*

`body` `content`

The body.

Example

rust

Calculate Fibonacci number using reccursive function

```rust
pub fn fibonacci_reccursive(n: i32) -> u64 {
    if n < 0 {
        panic!("{} is negative!", n);
        > to avoid negative numbers
    }
    match n {
        0 => panic!("zero is not a right argument to fibonacci_reccursive()!"),
        1 | 2 => 1,
        3 => 2,
        _ => fibonacci_reccursive(n - 1) + fibonacci_reccursive(n - 2),
        > 50 => 12586269025
    }
}
```